Odin: A Multi-Agent Contract Intelligence System for Risk Foresight

1. Context and Background

Companies sign many contracts with suppliers, landlords, employees, software vendors, and partners. Each contract contains deadlines, penalties, auto-renewals, and hidden risks. Contracts are usually read once and then forgotten, and the company pays for it later.

This problem calls for a team of AI agents rather than a single summarizer. Odin, the orchestrator, coordinates five specialized agents: Thor (extraction), the Norns (timeline), Loki (adversarial review), Heimdall (event monitoring), and Tyr (arbitration and action).

2. Problem Statement

Organizations manage large numbers of contracts with vendors, landlords, employees, software providers, and partners. The obligations inside them, such as renewal windows, payment schedules, penalty clauses, and jurisdiction terms, are rarely tracked after signing, and a missed obligation is usually discovered only after it has cost money. Contracts also do not exist in isolation: terms in one agreement can conflict with another, and real-world events such as late payments, outages, or regulatory changes continuously change which clauses matter. Current LLM-based tools treat this as a summarization task. Their output is a one-time, static description of a single document. It does not account for time, dependencies between contracts, or events that arrive after the contracts are read.

Challenge: Given a set of contracts and a stream of real-world events, build a multi-agent system that identifies obligations, anticipates what goes wrong if no action is taken, tests its own findings, and recommends concrete, evidence-backed actions.

Objective: To design, build, and evaluate a multi-agent contract intelligence system that produces a continuously updated, citation-grounded view of contractual risk and the actions needed to address it. A single-call contract summarizer does not meet this objective.

3. System Requirements

To address the aforementioned problem, the proposed system must satisfy the following functional and non-functional requirements.

3.1 Functional Requirements (FR)

FR1: Odin – Orchestrator

- **FR1.1 (Input):** Accept a company context and the set of contracts.
- **FR1.2 (Planning):** Build a work plan with one workstream per contract plus one cross-contract workstream.
- **FR1.3 (Delegation and Tracking):** Assign tasks to the other agents and track the status of every workstream.
- **FR1.4 (Coordination):** Manage the clarification and debate loops, decide when each contract's analysis is complete, and trigger the final synthesis.

FR2: Thor – Extraction Agent

- **FR2.1 (Extraction):** Extract parties, durations, renewal dates, payment schedules, obligations, penalty clauses, compliance requirements, and termination conditions.
- **FR2.2 (Evidence):** Attach a confidence score and a citation (clause number and supporting text) to every field.
- **FR2.3 (Ambiguity Handling):** Flag ambiguous, missing, or contradictory clauses instead of guessing.
- **FR2.4 (Re-extraction):** Re-extract a specific clause on request and save all output to the shared state store.

FR3: Norns – Timeline Agent

- **FR3.1 (Obligation Timeline):** Record past commitments and payments, compute what is due now, and project upcoming deadlines, renewals, and penalty triggers.
- **FR3.2 (Simulated Clock):** Support a clock that can move forward in time and recompute obligations at each position.
- **FR3.3 (Worst-Case Projection):** Produce a worst-case timeline if no action is taken, with financial exposure where the contracts provide the numbers.

FR4: Loki – Adversarial Review Agent

- **FR4.1 (Adversarial Review):** Read the contracts as the counterparty's lawyer and identify ambiguous clauses, one-sided terms, and hidden liabilities.
- **FR4.2 (Cross-Contract Conflicts):** Detect conflicts between different contracts.
- **FR4.3 (Evidence and Scoring):** Provide clause citations and a reasoned attack path for each finding, with a severity and likelihood estimate.
- **FR4.4 (Clarification Requests):** Request clarification from Thor when evidence is weak or extraction confidence is low.

FR5: Heimdall – Event Monitoring Agent

- **FR5.1 (Event Mapping):** Monitor the event feed, map each event to the affected contracts and clauses, and reassess exposure.
- **FR5.2 (Verification):** Verify or challenge Loki's claims against actual events and clause text.
- **FR5.3 (Alerts):** Raise severity-rated alerts for upcoming deadlines, renewals, payments, and potential breaches, while filtering out noise.

FR6: Tyr – Arbitration and Action Agent

- **FR6.1 (Rulings):** Resolve disagreements between Loki and Heimdall and record the rationale.
- **FR6.2 (Action Drafting):** Draft ready-to-send renewal, termination, or breach notices, amendment proposals, and escalation memos, citing the exact clauses.
- **FR6.3 (Human Approval):** Mark actions that need human approval, with a confidence level.

FR7: Agent Workflow

- **FR7.1 (Feedback Loops):** Loki must be able to send clarification requests to Thor, and Loki and Heimdall must be able to disagree, with Tyr resolving the dispute.

- **FR7.2 (Limits and Termination):** Loops must have configurable limits, and the system should stop when the evidence threshold is met or the budget is reached.

FR8: Synthesis

- **FR8.1 (Contract Report):** Generate an independent report per contract covering key terms, obligations, timeline, risks, and recommended actions, with citations and confidence levels.
- **FR8.2 (Company Risk Report):** Combine all contract reports into one report with cross-contract conflicts, total financial exposure, a prioritized action list, the worst-case projection, and a references section pointing to clause text.

FR9: Dashboard

- **FR9.1 (Contract Graph):** An interactive graph with contracts, clauses, and parties as nodes, and dependencies and conflicts as edges.
- **FR9.2 (Time Slider):** A slider that moves the simulated clock and updates deadlines, risks, and penalties live.
- **FR9.3 (Exposure Views):** A financial exposure gauge and a risk heatmap by contract, clause type, and month.
- **FR9.4 (Agent Debate Panel):** A live view of Loki's attacks, Heimdall's verifications, and Tyr's rulings.
- **FR9.5 (Alert Feed and Action Panel):** A prioritized alert stream and a panel of drafted notices with clause citations.

3.2 Non-Functional Requirements (NFR)

NFR1: Architecture

- Each agent must have a clearly separated responsibility and its own prompts, tools, or both.
- Agents must communicate through a defined state and message structure.
- Either a state-machine workflow or a group-chat multi-agent design is acceptable.

NFR2: Analysis Quality

- Every extracted term must cite its source text, and hallucinated clauses are penalized.
- Ambiguity and conflicting interpretations must be flagged explicitly, not silently resolved.
- Sourced facts must be clearly distinguished from agent-generated analysis.

NFR3: Reliability

- The system must handle malformed contracts, failed API calls, and unparseable events gracefully, with retries and fallbacks.
- The system must prevent infinite loops between agents.
- The system must never report an analysis as complete when it is not.

NFR4: Observability

- The system must expose the work plan and per-contract status, extractions and confidence scores, timeline projections, attack proposals and verifications, rulings and rationale, agent-to-agent transitions, and the final synthesis.

NFR5: Performance

- The system must avoid duplicate agent calls and redundant re-extraction, preferring targeted clarification over re-reading whole contracts.
- The system must provide configurable limits for maximum iterations, agent calls, and token/API usage.

NFR6: Reproducibility

- State must be stored in an explicit, structured shared store, and intermediate outputs must not exist only inside hidden prompts.
- The same input and configuration should give reasonably reproducible results.

NFR7: LLM Usage

- The LLM must be a meaningful part of the agent system; sending all contracts to a single call for a summary does not satisfy the multi-agent requirement.

3. Conclusion

Meeting these requirements will produce a system that does more than summarize contracts: it extracts obligations with evidence, projects them through time, tests them adversarially, and keeps them current against incoming events. By coordinating specialized agents and presenting citation-grounded reports, actions, and a clear dashboard, Odin demonstrates a practical approach to contract risk management.